

DSAA2043: Exercise 7

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed arrays are accepted. However, when using Python, array indices must start from 0.

Problem 1. Consider the directed weighted graph G in Figure 1. All edge weights are non-negative. We take s as the source vertex.

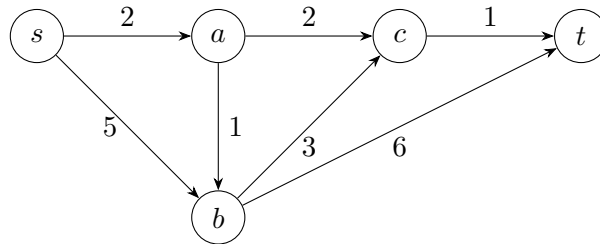


Figure 1: The directed weighted graph for Problem 1.

Q1.1. Run Dijkstra's algorithm on this graph with source s . Show the step-by-step execution. At each step, indicate:

- which vertex is selected into the settled set, and
- how each outgoing edge from this vertex is relaxed (show the distance updates).

Q1.2. Run the Bellman-Ford algorithm on the same graph with source s . Show the step-by-step execution. For each iteration, show the distance estimates after processing all edges and indicate which relaxations changed a distance.

Answer:

A1.1. Dijkstra's algorithm from s .

We maintain for each vertex v a tentative distance $d(v)$ and a set S of vertices whose final shortest-path distances are already known.

Initially:

$$d(s) = 0, \quad d(a) = d(b) = d(c) = d(t) = \infty, \quad S = \emptyset.$$

Step 1. The vertex with minimum tentative distance is s ; select s into S :

$$S = \{s\}.$$

Relax outgoing edges of s :

- Edge (s, a) with weight 2: $d(a)$ changes from ∞ to $0 + 2 = 2$.
- Edge (s, b) with weight 5: $d(b)$ changes from ∞ to $0 + 5 = 5$.

Now:

$$d(s) = 0, \quad d(a) = 2, \quad d(b) = 5, \quad d(c) = \infty, \quad d(t) = \infty.$$

Step 2. Among vertices not in S , the smallest tentative distance is $d(a) = 2$; select a :

$$S = \{s, a\}.$$

Relax outgoing edges of a :

- Edge (a, b) with weight 1: $d(b)$ changes from 5 to $\min\{5, 2 + 1\} = 3$.
- Edge (a, c) with weight 2: $d(c)$ changes from ∞ to $2 + 2 = 4$.

Now:

$$d(s) = 0, \quad d(a) = 2, \quad d(b) = 3, \quad d(c) = 4, \quad d(t) = \infty.$$

Step 3. The smallest tentative distance among vertices not in S is $d(b) = 3$; select b :

$$S = \{s, a, b\}.$$

Relax outgoing edges of b :

- Edge (b, c) with weight 3: current $d(c) = 4$, candidate $3 + 3 = 6$, no change.
- Edge (b, t) with weight 6: $d(t)$ changes from ∞ to $3 + 6 = 9$.

Now:

$$d(s) = 0, \quad d(a) = 2, \quad d(b) = 3, \quad d(c) = 4, \quad d(t) = 9.$$

Step 4. The smallest tentative distance among vertices not in S is $d(c) = 4$; select c :

$$S = \{s, a, b, c\}.$$

Relax outgoing edge of c :

- Edge (c, t) with weight 1: $d(t)$ changes from 9 to $\min\{9, 4 + 1\} = 5$.

Now:

$$d(s) = 0, \quad d(a) = 2, \quad d(b) = 3, \quad d(c) = 4, \quad d(t) = 5.$$

Step 5. The only remaining vertex is t with $d(t) = 5$; select t :

$$S = \{s, a, b, c, t\}.$$

No outgoing edges from t , so we are done.

Final shortest-path distances from s are:

$$d(s) = 0, \quad d(a) = 2, \quad d(b) = 3, \quad d(c) = 4, \quad d(t) = 5.$$

A1.2. Bellman–Ford algorithm from s .

Method 1 (single-array in-place relaxation, as in L10, slides 52–53).

Let the vertices be ordered as (s, a, b, c, t) and list all directed edges:

$$(s, a, 2), \quad (s, b, 5), \quad (a, b, 1), \quad (a, c, 2), \quad (b, c, 3), \quad (b, t, 6), \quad (c, t, 1).$$

We do $|V| - 1 = 4$ iterations. Distances are initialized as

$$d(s) = 0, \quad d(a) = d(b) = d(c) = d(t) = \infty.$$

Iteration 1. Process edges in the above order.

- $(s, a, 2)$: $d(a)$ changes $\infty \rightarrow 0 + 2 = 2$.
- $(s, b, 5)$: $d(b)$ changes $\infty \rightarrow 0 + 5 = 5$.
- $(a, b, 1)$: $d(b)$ changes $5 \rightarrow \min\{5, 2 + 1\} = 3$.
- $(a, c, 2)$: $d(c)$ changes $\infty \rightarrow 2 + 2 = 4$.
- $(b, c, 3)$: candidate $3 + 3 = 6$, current $d(c) = 4$, no change.
- $(b, t, 6)$: $d(t)$ changes $\infty \rightarrow 3 + 6 = 9$.
- $(c, t, 1)$: $d(t)$ changes $9 \rightarrow \min\{9, 4 + 1\} = 5$.

After iteration 1:

$$d(s) = 0, d(a) = 2, d(b) = 3, d(c) = 4, d(t) = 5.$$

Iteration 2. Process all edges again.

- $(s, a, 2)$: candidate $0 + 2 = 2$, no change.
- $(s, b, 5)$: candidate $0 + 5 = 5$, current $d(b) = 3$, no change.
- $(a, b, 1)$: candidate $2 + 1 = 3$, no change.
- $(a, c, 2)$: candidate $2 + 2 = 4$, no change.
- $(b, c, 3)$: candidate $3 + 3 = 6$, no change.
- $(b, t, 6)$: candidate $3 + 6 = 9$, current $d(t) = 5$, no change.
- $(c, t, 1)$: candidate $4 + 1 = 5$, no change.

Distances remain

$$d(s) = 0, d(a) = 2, d(b) = 3, d(c) = 4, d(t) = 5.$$

Iterations 3 and 4. Repeating the relaxation over all edges makes no further changes (all candidate distances are no smaller than the current ones). Thus after $|V| - 1$ iterations, Bellman–Ford also ends with

$$d(s) = 0, d(a) = 2, d(b) = 3, d(c) = 4, d(t) = 5,$$

which agrees with Dijkstra’s result.

Method 2 (iteration table $d^{(i)}$).

For Bellman–Ford, we explicitly distinguish the distance array in each iteration. Let $d^{(i)}(v)$ be the distance estimate of vertex v after iteration i . Conceptually this gives a sequence of arrays $d^{(0)}, d^{(1)}, \dots$ as in the lecture slides (Lecture L10, slide 55). We always compute $d^{(i+1)}$ only from $d^{(i)}$.

List vertices as (s, a, b, c, t) and all directed edges as

$$(s, a, 2), (s, b, 5), (a, b, 1), (a, c, 2), (b, c, 3), (b, t, 6), (c, t, 1).$$

We perform $|V| - 1 = 4$ iterations.

Initialization.

$$d^{(0)}(s) = 0, \quad d^{(0)}(a) = d^{(0)}(b) = d^{(0)}(c) = d^{(0)}(t) = \infty.$$

Iteration 1. We start from $d^{(0)}$ and compute $d^{(1)}$; initially set $d^{(1)}(v) \leftarrow d^{(0)}(v)$ for all v , then process each edge (u, v, w) and update

$$d^{(1)}(v) \leftarrow \min(d^{(1)}(v), d^{(0)}(u) + w).$$

Processing edges in order:

- $(s, a, 2)$: $d^{(1)}(a)$ changes $\infty \rightarrow 0 + 2 = 2$.
- $(s, b, 5)$: $d^{(1)}(b)$ changes $\infty \rightarrow 0 + 5 = 5$.
- $(a, b, 1)$: using $d^{(0)}(a) = \infty$ gives no improvement, so no change here.
- $(a, c, 2)$: using $d^{(0)}(a) = \infty$ gives no improvement, no change.
- $(b, c, 3)$: using $d^{(0)}(b) = \infty$ gives no improvement, no change.
- $(b, t, 6)$: using $d^{(0)}(b) = \infty$ gives no improvement, no change.
- $(c, t, 1)$: using $d^{(0)}(c) = \infty$ gives no improvement, no change.

After iteration 1:

$$d^{(1)}(s) = 0, \quad d^{(1)}(a) = 2, \quad d^{(1)}(b) = 5, \quad d^{(1)}(c) = \infty, \quad d^{(1)}(t) = \infty.$$

Iteration 2. Now compute $d^{(2)}$ from $d^{(1)}$ in the same way.

- Initialize $d^{(2)}(v) \leftarrow d^{(1)}(v)$ for all v .
- $(s, a, 2)$: $d^{(1)}(s) + 2 = 2$, no improvement over $d^{(2)}(a) = 2$.
- $(s, b, 5)$: candidate $0 + 5 = 5$, no improvement over $d^{(2)}(b) = 5$.
- $(a, b, 1)$: $d^{(1)}(a) + 1 = 2 + 1 = 3 < 5$, so $d^{(2)}(b)$ changes $5 \rightarrow 3$.
- $(a, c, 2)$: $d^{(1)}(a) + 2 = 2 + 2 = 4$, so $d^{(2)}(c)$ changes $\infty \rightarrow 4$.
- $(b, c, 3)$: use $d^{(1)}(b) = 5$, candidate $5 + 3 = 8$, no improvement over $d^{(2)}(c) = 4$.
- $(b, t, 6)$: use $d^{(1)}(b) = 5$, candidate $5 + 6 = 11$, so $d^{(2)}(t)$ changes $\infty \rightarrow 11$.
- $(c, t, 1)$: use $d^{(1)}(c) = \infty$, no improvement.

After iteration 2:

$$d^{(2)}(s) = 0, \quad d^{(2)}(a) = 2, \quad d^{(2)}(b) = 3, \quad d^{(2)}(c) = 4, \quad d^{(2)}(t) = 11.$$

Iteration 3. Compute $d^{(3)}$ from $d^{(2)}$:

- Initialize $d^{(3)}(v) \leftarrow d^{(2)}(v)$.
- $(s, a, 2), (s, b, 5)$: no improvements.

- $(a, b, 1)$, $(a, c, 2)$: candidates are 3 and 4, no improvements.
- $(b, c, 3)$: candidate $d^{(2)}(b) + 3 = 3 + 3 = 6$, no improvement over 4.
- $(b, t, 6)$: candidate $3 + 6 = 9 < 11$, so $d^{(3)}(t)$ changes $11 \rightarrow 9$.
- $(c, t, 1)$: candidate $d^{(2)}(c) + 1 = 4 + 1 = 5 < 9$, so $d^{(3)}(t)$ changes $9 \rightarrow 5$.

After iteration 3:

$$d^{(3)}(s) = 0, d^{(3)}(a) = 2, d^{(3)}(b) = 3, d^{(3)}(c) = 4, d^{(3)}(t) = 5.$$

Iteration 4. Compute $d^{(4)}$ from $d^{(3)}$. All candidates $d^{(3)}(u) + w(u, v)$ are no better than the current values, so no distances change and

$$d^{(4)}(v) = d^{(3)}(v) \quad \text{for all } v.$$

Thus after $|V| - 1 = 4$ iterations, Bellman–Ford ends with

$$d^{(4)}(s) = 0, d^{(4)}(a) = 2, d^{(4)}(b) = 3, d^{(4)}(c) = 4, d^{(4)}(t) = 5,$$

which agrees with the result of Dijkstra’s algorithm.

Problem 2. Problem 3. Consider a directed simple graph $G = (V, E)$ where each edge $e \in E$ has an arbitrary weight $w(e)$ (which can be negative). It is known that G does not have negative cycles. Prove: given any vertices $s, t \in V$, at least one shortest path from s to t is a simple path (i.e., no vertex appears twice on the path).

Remark: This implies that the path must have at most $|V| - 1$ edges.

Solution. Let π be a shortest path from s to t that uses the least number of edges. We will prove that π must be a simple path. Let us list all the vertices on the path from s to t as $u_1, u_2, \dots, u_t$, where $u_1 = s$, $u_t = t$, and $t - 1$ is the number of edges on π . If π is not a simple path, then there must exist $1 \leq i < j \leq t$ with $u_i = u_j$. Thus, the sub-path $u_i, u_{i+1}, \dots, u_{j-1}, u_j$ is a cycle. The length of the cycle must be non-negative. By removing this sub-path, we obtain another path π' from s to t : $u_1, u_2, \dots, u_i, u_{j+1}, u_{j+2}, \dots, u_t$. The new path π' cannot have a length greater than π , but uses strictly fewer edges. This contradicts the definition of π .